ОТЧЁТ

по лабораторной работе №12

«Работа с реляционными базами данных через SQLAlchemy»

Вариант 20. Система управления задачами

1. Цель работы

Научиться описывать таблицы классами SQLAlchemy, связывать пользователя, проект и задачу, выполнять создание, чтение, изменение, удаление, поиск, фильтрацию, сортировку и транзакцию через сессию.

2. Постановка индивидуального задания

Вариант 20. Сущности User, Project и Task. Нужно создавать задачи и назначать их пользователям. Проект принадлежит владельцу, задача относится к проекту и может иметь исполнителя.

База — файл SQLite. Создание проекта вместе с первой задачей выполняется одной транзакцией: незавершённая пара не сохраняется.

3. Краткие теоретические сведения

ORM сопоставляет класс строке таблицы. Программист работает с объектами, а SQLAlchemy строит SQL. Сессия отслеживает изменения и фиксирует их методом commit. relationship описывает связь: у пользователя есть проекты, у проекта — задачи, у задачи — исполнитель.

Фильтр и сортировка задаются в запросе select, а не циклом по всем строкам после загрузки. Транзакция нужна, когда несколько объектов должны появиться вместе.

4. Описание программы

Модели User, Project и Task связаны внешними ключами. При первом запуске создаются два пользователя, два проекта и две задачи. Меню показывает пользователей с числом проектов, задачи внутри проекта, общий список, поиск по фрагменту названия, фильтр по статусу и сортировку.

Назначение исполнителя меняет и ссылку, и статус задачи на «назначена». Тесты проверяют связи после наполнения, фильтр, поиск, атомарное создание проекта и назначение.

5. Исходный код программы

Ниже приведён исходный код разработанных файлов.

Листинг — main.py

```
"""
Лабораторная работа №12. Вариант 20.
Система задач на SQLAlchemy.
"""

import sys

from service import TaskService, make_engine, seed
from sqlalchemy.orm import Session

if sys.platform == "win32":
    sys.stdout.reconfigure(encoding="utf-8")
    sys.stderr.reconfigure(encoding="utf-8")
    if not sys.stdin.isatty():
        sys.stdin.reconfigure(encoding="utf-8")



def read_line(prompt):
    if sys.stdin.isatty():
        return input(prompt).strip()
    raw = sys.stdin.readline()
    if raw == "":
        raise EOFError
    text = raw.rstrip("\r\n")
    print(f"{prompt}{text}")
    return text.strip()



def show_tasks(tasks):
    if not tasks:
        print("Задач нет.")
        return
    print(f"{'ID':<5}{'Задача':<24}{'Статус':<14}{'Проект':<22}{'Исполнитель'}")
    for task in tasks:
        assignee = task.assignee.name if task.assignee else "—"
        print(f"{task.id:<5}{task.title:<24}{task.status:<14}{task.project.title:<22}{assignee}")



def print_menu():
    print()
    print("===== Задачи (SQLAlchemy) =====")
    print("1. Пользователи")
    print("2. Проекты и задачи владельца")
    print("3. Все задачи")
    print("4. Создать пользователя")
    print("5. Создать проект с первой задачей")
    print("6. Назначить исполнителя")
    print("7. Поиск по названию")
    print("8. Фильтр по статусу")
    print("9. Сортировка по статусу")
    print("0. Выход")
    print()



def main():
    engine = make_engine()
    with Session(engine) as session:
        seed(session)
        service = TaskService(session)
        print("База tasks.db готова, связи User — Project — Task загружены.")
        while True:
            print_menu()
            choice = read_line("Выберите действие: ")
            try:
                if choice == "1":
                    for user in service.users():
                        print(f"#{user.id} {user.name} <{user.email}>, проектов: {len(user.projects)}")
                elif choice == "2":
```

```
                    for project in service.projects():
                        print(f"\n{project.title} — владелец {project.owner.name}")
                        for task in project.tasks:
                            print(f"  - {task.title} [{task.status}]")
                elif choice == "3":
                    show_tasks(service.tasks())
                elif choice == "4":
                    user = service.create_user(read_line("Имя: "), read_line("Email: "))
                    print(f"Создан пользователь #{user.id}")
                elif choice == "5":
                    project = service.create_project_with_task(
                        read_line("Email владельца: "),
                        read_line("Название проекта: "),
                        read_line("Первая задача: "),
                    )
                    print(f"Проект #{project.id} и задача сохранены одной транзакцией.")
                elif choice == "6":
                    task = service.assign(int(read_line("ID задачи: ")), read_line("Email исполнителя: "))
                    print(f"Задача «{task.title}» назначена, статус: {task.status}")
                elif choice == "7":
                    show_tasks(service.find_tasks(read_line("Фрагмент названия: ")))
                elif choice == "8":
                    show_tasks(service.tasks(status=read_line("Статус: ")))
                elif choice == "9":
                    show_tasks(service.tasks(order="status"))
                elif choice == "0":
                    print("Работа программы завершена.")
                    break
                else:
                    print("Ошибка: выберите пункт от 0 до 9.")
            except ValueError as exc:
                print(f"Ошибка: {exc}")
            except EOFError:
                break


if __name__ == "__main__":
    main()
```

Листинг — models.py

```
"""ORM-модели системы задач."""

from sqlalchemy import ForeignKey, String
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship


class Base(DeclarativeBase):
    pass


class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    email: Mapped[str] = mapped_column(String(120), unique=True)
    projects: Mapped[list["Project"]] = relationship(back_populates="owner")
    tasks: Mapped[list["Task"]] = relationship(back_populates="assignee")


class Project(Base):
    __tablename__ = "projects"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(150))
    owner_id: Mapped[int] = mapped_column(ForeignKey("users.id"))
    owner: Mapped[User] = relationship(back_populates="projects")
    tasks: Mapped[list["Task"]] = relationship(back_populates="project", cascade="all, delete-orphan")


class Task(Base):
    __tablename__ = "tasks"
```

```
    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(150))
    status: Mapped[str] = mapped_column(String(30), default="новая")
    project_id: Mapped[int] = mapped_column(ForeignKey("projects.id"))
    assignee_id: Mapped[int | None] = mapped_column(ForeignKey("users.id"), nullable=True)
    project: Mapped[Project] = relationship(back_populates="tasks")
    assignee: Mapped[User | None] = relationship(back_populates="tasks")
```

Листинг — service.py

```
"""Операции SQLAlchemy. Назначение задачи меняет задачу и проект в одной транзакции."""

import os

from sqlalchemy import create_engine, select
from sqlalchemy.orm import Session

from models import Base, Project, Task, User

BASE = os.path.dirname(os.path.abspath(__file__))
DB_PATH = os.path.join(BASE, "data", "tasks.db")



def make_engine(path=None):
    target = path or DB_PATH
    folder = os.path.dirname(target)
    if folder:
        os.makedirs(folder, exist_ok=True)
    engine = create_engine(f"sqlite:///{target}", echo=False)
    Base.metadata.create_all(engine)
    return engine



def seed(session: Session):
    if session.scalar(select(User).limit(1)):
        return
    ivan = User(name="Иванов", email="ivan@example.com")
    anna = User(name="Петрова", email="anna@example.com")
    session.add_all([ivan, anna])
    session.flush()
    site = Project(title="Сайт провайдера", owner=ivan)
    billing = Project(title="Биллинг", owner=anna)
    session.add_all([site, billing])
    session.flush()
    session.add(Task(title="Макет главной", status="новая", project=site, assignee=anna))
    session.add(Task(title="Оплата тарифа", status="в работе", project=billing, assignee=ivan))
    session.commit()



class TaskService:
    def __init__(self, session: Session):
        self.session = session

    def users(self):
        return list(self.session.scalars(select(User).order_by(User.name)))

    def projects(self):
        return list(self.session.scalars(select(Project).order_by(Project.title)))

    def tasks(self, status=None, project_title=None, order="title"):
        query = select(Task)
        if status:
            query = query.where(Task.status == status)
        if project_title:
            query = query.join(Project).where(Project.title == project_title)
        if order == "status":
            query = query.order_by(Task.status, Task.title)
        else:
            query = query.order_by(Task.title)
        return list(self.session.scalars(query))
```

```
    def create_user(self, name, email):
        user = User(name=name, email=email)
        self.session.add(user)
        self.session.commit()
        return user

    def create_project_with_task(self, owner_email, project_title, task_title):
        """Атомарно: проект без первой задачи не сохраняется."""
        owner = self.session.scalar(select(User).where(User.email == owner_email))
        if owner is None:
            raise ValueError("пользователь не найден")
        try:
            project = Project(title=project_title, owner=owner)
            task = Task(title=task_title, status="новая", project=project, assignee=owner)
            self.session.add(project)
            self.session.add(task)
            self.session.commit()
        except Exception:
            self.session.rollback()
            raise
        return project

    def assign(self, task_id, email):
        task = self.session.get(Task, task_id)
        user = self.session.scalar(select(User).where(User.email == email))
        if task is None or user is None:
            raise ValueError("задача или пользователь не найдены")
        task.assignee = user
        task.status = "назначена"
        self.session.commit()
        return task

    def find_tasks(self, text):
        return list(
            self.session.scalars(select(Task).where(Task.title.contains(text)).order_by(Task.title))
        )
```

Листинг — tests\test_orm.py

```
import os
import sys

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from service import TaskService, make_engine, seed
from sqlalchemy.orm import Session



def service(tmp_path):
    engine = make_engine(str(tmp_path / "t.db"))
    session = Session(engine)
    seed(session)
    return TaskService(session), session



def test_seed_relations(tmp_path):
    svc, session = service(tmp_path)
    projects = svc.projects()
    assert projects[0].owner.name
    assert projects[0].tasks
    session.close()



def test_filter_and_sort(tmp_path):
    svc, session = service(tmp_path)
    found = svc.tasks(status="в работе")
    assert len(found) == 1
    ordered = svc.tasks(order="status")
    assert ordered
    session.close()



def test_search(tmp_path):
```

```
    svc, session = service(tmp_path)
    assert svc.find_tasks("Оплата")
    session.close()


def test_transaction_creates_task(tmp_path):
    svc, session = service(tmp_path)
    project = svc.create_project_with_task("ivan@example.com", "API", "Контракт")
    assert project.tasks[0].title == "Контракт"
    session.close()


def test_assign(tmp_path):
    svc, session = service(tmp_path)
    task = svc.tasks()[0]
    updated = svc.assign(task.id, "anna@example.com")
    assert updated.status == "назначена"
    assert updated.assignee.email == "anna@example.com"
    session.close()
```

6. Примеры работы программы

Снимки показывают начальные связи и операции создания проекта, назначения, фильтра и поиска.

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python12> python main.py

База tasks.db готова, связи User — Project — Task загружены.

===== Задачи (SQLAlchemy) =====
1. Пользователи
2. Проекты и задачи владельца
3. Все задачи
4. Создать пользователя
5. Создать проект с первой задачей
6. Назначить исполнителя
7. Поиск по названию
8. Фильтр по статусу
9. Сортировка по статусу
0. Выход

Выберите действие: 1
#1 Иванов <ivan@example.com>, проектов: 1
#2 Петрова <anna@example.com>, проектов: 1

===== Задачи (SQLAlchemy) =====
1. Пользователи
2. Проекты и задачи владельца
3. Все задачи
4. Создать пользователя
5. Создать проект с первой задачей
6. Назначить исполнителя
7. Поиск по названию
8. Фильтр по статусу
9. Сортировка по статусу
0. Выход

Выберите действие: 2

Биллинг — владелец Петрова
  - Оплата тарифа [в работе]

Сайт провайдера — владелец Иванов
  - Макет главной [новая]

===== Задачи (SQLAlchemy) =====
1. Пользователи
2. Проекты и задачи владельца
3. Все задачи
4. Создать пользователя
5. Создать проект с первой задачей
6. Назначить исполнителя
7. Поиск по названию
8. Фильтр по статусу
9. Сортировка по статусу
0. Выход

Выберите действие: 3
ID   Задача                Статус       Проект               Исполнитель
1    Макет главной         новая        Сайт провайдера      Петрова
2    Оплата тарифа         в работе     Биллинг              Иванов
```

Рисунок 1 – Пользователи, проекты и задачи, загруженные через ORM

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python12> python main.py

База tasks.db готова, связи User — Project — Task загружены.

===== Задачи (SQLAlchemy) =====
1. Пользователи
2. Проекты и задачи владельца
3. Все задачи
4. Создать пользователя
5. Создать проект с первой задачей
6. Назначить исполнителя
7. Поиск по названию
8. Фильтр по статусу
9. Сортировка по статусу
0. Выход

Выберите действие: 5
Email владельца: ivan@example.com
Название проекта: Личный кабинет
Первая задача: Форма входа
Проект #3 и задача сохранены одной транзакцией.

===== Задачи (SQLAlchemy) =====
1. Пользователи
2. Проекты и задачи владельца
3. Все задачи
4. Создать пользователя
5. Создать проект с первой задачей
6. Назначить исполнителя
7. Поиск по названию
8. Фильтр по статусу
9. Сортировка по статусу
0. Выход

Выберите действие: 6
ID задачи: 1
Email исполнителя: anna@example.com
Задача «Макет главной» назначена, статус: назначена

===== Задачи (SQLAlchemy) =====
1. Пользователи
2. Проекты и задачи владельца
3. Все задачи
4. Создать пользователя
5. Создать проект с первой задачей
6. Назначить исполнителя
7. Поиск по названию
8. Фильтр по статусу
9. Сортировка по статусу
0. Выход

Выберите действие: 8
Статус: в работе
ID   Задача                 Статус        Проект                Исполнитель
2    Оплата тарифа          в работе      Биллинг               Иванов

===== Задачи (SQLAlchemy) =====
```

Рисунок 2 – Новый проект с задачей, назначение исполнителя, фильтр и поиск

7. Ответы на контрольные вопросы

1. Что делает ORM?

Переводит объекты Python в строки таблиц и обратно, чтобы прикладной код не собирал каждый SQL вручную.

2. Зачем сессия?

Она помнит новые и изменённые объекты и одной транзакцией записывает их в базу по commit.

3. Как задана связь проекта и задач?

У Project есть коллекция tasks, у Task — ссылка project. Внешний ключ project_id хранится в таблице задач.

4. Где здесь транзакция?

Проект и его первая задача добавляются в одну сессию. commit фиксирует оба объекта, rollback отменяет оба.

5. Чем фильтр ORM отличается от фильтра списка?

Условие уходит в запрос к базе. Не нужно сначала читать все задачи в память.

8. Вывод

В лабораторной работе №12 система задач варианта 20 хранится в SQLite через SQLAlchemy. Пользователь, проект и задача связаны, меню выполняет поиск, фильтр, сортировку и назначение, а создание проекта с первой задачей проходит одной транзакцией.